SUBNET / TECHNICAL BRIEF

Offline-first peer discovery and chat

Current implementation, network architecture, observed failure modes, and the recommended path to automatic discovery.

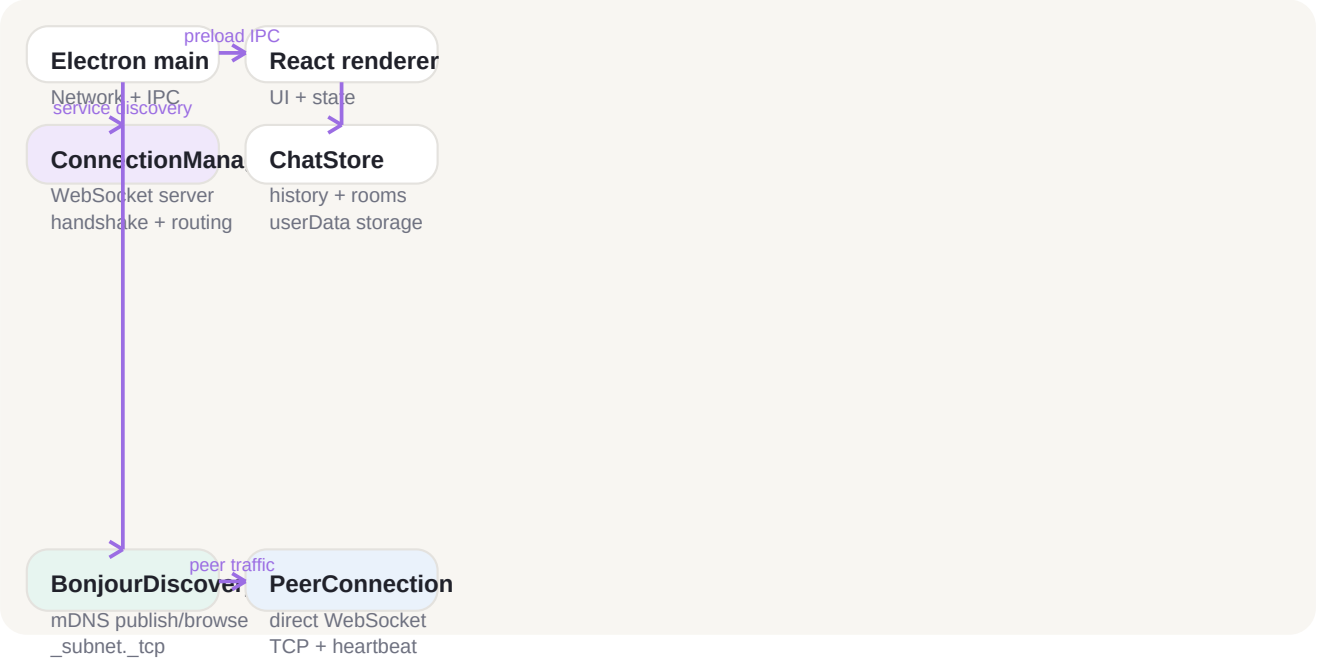
System goal
A macOS Electron application that discovers nearby Subnet users, establishes direct peer connections, and supports private chats, rooms, invitations, attachments, images, and video messages over a local network.

Application	Subnet 0.1.0
Runtime	Electron + React + TypeScript + Vite
Transport	WebSocket over TCP
Discovery	Bonjour / mDNS service discovery
Storage	Local persistent chat store in Electron userData

Prepared for technical discussion

1. Current architecture

Subnet is a local, decentralized messaging system. Each running app is both a client and a server: it advertises itself, listens for inbound connections, discovers other instances, and maintains its own local state.



Runtime layers

- Electron main process: owns networking, OS permissions, media/file operations, persistent storage, and native IPC handlers.
- React renderer: owns application state, sidebar, radar, chat views, room views, profile/settings UI, notifications, and media modals.
- Preload bridge: exposes a constrained window.subnet API to the renderer instead of exposing Node.js directly.
- ConnectionManager: starts an ephemeral WebSocket server, accepts inbound sockets, initiates outbound sockets, performs handshake and duplicate-connection tie-breaking, and routes messages.
- BonjourDiscovery: publishes and browses _subnet._tcp.local for users and _subnet-room._tcp.local for anonymous rooms.
- ChatStore and FileTransferManager: persist conversations/rooms and transfer attachments in chunks with metadata and integrity information.

Connection lifecycle

- 1. Start WebSocket server on port 0; the OS assigns an available ephemeral port.
- 2. Detect the local interface address and publish a Bonjour service containing the service name and port.
- 3. Browse for peer services and room services using multicast DNS.
- 4. On discovery, attempt a direct WebSocket connection and exchange a handshake containing peer ID, display name, avatar metadata, protocol version, and timestamp.
- 5. Maintain liveness with heartbeat ping/pong messages; remove stale connections after timeout.
- 6. Route chat, invitation, room, profile-update, and file-transfer protocol envelopes through the verified connection.

2. Implemented protocol and product capabilities

The application protocol is versioned and uses typed message envelopes. The current protocol includes direct messages, invitations, rooms, media metadata, file offers, chunk requests, acknowledgements, profile updates, room membership updates, and room closure events.

Area	Technical implementation	Purpose
Identity	Stable peer ID + display name + avatar metadata	Identify a device and keep profile state consistent
Discovery	Bonjour service types _subnet._tcp and _subnet-room._tcp	Find users and rooms without a directory
Transport	Direct WebSocket/TCP with ephemeral listening port	Exchange chat data peer-to-peer
Reliability	Handshake timeout, heartbeat, duplicate tie-break, reconnect/lost-peer handling	Avoid duplicate sockets and stale online state
Files	File offer/request/chunk/complete/cancel flow, SHA-256 metadata, thumbnails	Transfer attachments and media safely
Persistence	Electron userData store with conversation and room history	Retain local chat history across restarts

Application capabilities currently represented in the UI and protocol

- Private one-to-one conversations with invitation/accept/decline flow and local history.
- Private rooms with invitations, anonymous rooms with join flow, member updates, add-member flow, leave-room and host close-room actions.
- Local profile controls: discoverability, avatar sharing, custom or preset avatars, camera/microphone permission state, and network details.
- Radar-style nearby-user visualization driven by discovered peer state.
- Attachments, image previews, video capture, media transfer, download/save actions, and full-size media viewing.
- Native macOS configuration: local-network, Bonjour, camera, and microphone usage descriptions plus macOS entitlements.

Important boundary

The current design is LAN-only and decentralized. It does not have a central directory, public rendezvous service, NAT traversal, or relay fallback. Therefore, discovery depends on the local network carrying Bonjour/mDNS multicast.

3. Network problem observed

Two Macs were connected to the same campus network and could reach each other with unicast IP traffic, but the Subnet radar displayed zero users. The tests separate the problem into transport reachability and discovery reachability.

Test	Result	Interpretation
IP addresses	10.7.13.223 and 10.7.9.167	Different host addresses; not a problem by itself
Subnet mask	255.255.224.0 (/19)	Both addresses are in 10.7.0.0/19
Ping	0% packet loss in both directions	ICMP reachability works
Netcat	TCP connection to the current peer port succeeds	Unicast TCP/WebSocket path is reachable
Local listener	lsof shows Subnet listening on *:61556	The application is bound to network interfaces
Bonjour browse	Each Mac sees only its own _subnet._tcp service	mDNS multicast discovery is filtered or isolated

Diagnosis
The campus network allows unicast traffic between the Macs but does not allow the multicast DNS advertisements used by Bonjour. The application can connect if it already knows the correct IP address and current port, but it cannot automatically learn the peer from mDNS.

Why the port sometimes appeared wrong

Subnet intentionally requests port 0, so every running instance receives a different ephemeral port. A previously displayed port can become stale after restart. The correct port must be read from the current app instance or current Bonjour record.

Why similar IP prefixes are not enough

Matching 10.7 or sharing a /19 only indicates address grouping. It does not guarantee that multicast packets, TCP connections, client-to-client traffic, or firewall rules are permitted. Network policy controls those paths independently.

Likely network causes

- Wireless client isolation or peer-to-peer isolation on the access point.
- Bonjour/mDNS multicast filtering across VLANs or access points.
- Firewall or inbound application permission blocking the Electron bundle.
- Different Wi-Fi security zones even when the visible SSID or IP range appears similar.
- Stale port information after an application restart.

4. Automatic discovery options

The current LAN-only design cannot guarantee automatic discovery on a network that blocks Bonjour. The following architectures remove that dependency to different degrees.

Option	Architecture	Offline on campus LAN?	Trade-off
A	Campus-local rendezvous server: apps register over HTTPS; server returns peer candidates; chat remains direct WebSocket/TCP.	Yes, if server is reachable on campus	Requires a maintained local server and known hostname/IP
B	Public rendezvous + STUN/TURN: server exchanges candidates; TURN relays traffic when direct P2P fails.	No, requires internet	Infrastructure cost; relay traffic is no longer LAN-only
C	Network fix: enable mDNS/Bonjour reflection or remove client isolation across the relevant VLANs.	Yes	Requires campus network administrator
D	QR/deep-link invite containing IP, port, or token.	Yes	Manual step; ephemeral ports and IP changes remain

Recommended target architecture

- Use a campus-local rendezvous service if the goal is operation without public internet.
- On app start, register a stable device ID, display name, avatar metadata, and current connectivity candidates over HTTPS.
- Return peer candidates to both clients; attempt direct WebSocket/TCP first.
- Use a stable subnet://invite/ deep link or HTTPS link that opens the app and resolves the token through the rendezvous service.
- Keep a QR/IP fallback for environments where the rendezvous service is unavailable.
- If future requirements include home networks or different campuses, add ICE/STUN/TURN or a secure relay layer.

Clarification about HTTPS

HTTPS does not inherently mean public internet. An HTTPS server can run on a campus LAN and remain reachable when the internet is unavailable. The requirement is a stable, reachable server address. A localhost link is not a solution because localhost always means the computer opening the link.

5. Discussion checklist and test plan

Questions for the teacher or network administrator

- Are wireless clients allowed to communicate directly across access points and VLANs?
- Is Bonjour/mDNS multicast allowed or reflected between the relevant Wi-Fi segments?
- Can a small campus-local HTTPS rendezvous service be deployed with a stable DNS name?
- Are inbound TCP connections from student devices allowed, and which port range is permitted?
- Should the final product remain LAN-only, or is an internet relay acceptable for unavailable direct paths?

Current diagnostic commands

```
networksetup -getinfo Wi-Fi
ping -c 4 <other-device-ip>
dns-sd -B _subnet._tcp local.
lsof -nP -iTCP:<current-port> -sTCP:LISTEN
nc -vz <other-device-ip> <current-port>
```

Interpretation matrix

Ping	TCP	mDNS	Meaning
Pass	Pass	Pass	Current architecture should discover and connect
Pass	Pass	Fail	Discovery failure; rendezvous server, QR, or network fix required
Pass	Fail	Any	Firewall, port, process, or application binding problem
Fail	Fail	Any	Network isolation, wrong network, or routing problem

Security and permission constraints

- macOS Local Network permission must be approved by the user; an application cannot silently grant it.
- Camera and microphone permissions are separate from network discovery and are required only for media capture.
- A relay architecture changes the privacy boundary because message data may transit infrastructure outside the local network.
- Stable peer IDs should be separated from IP addresses; IP addresses and ports are transient transport details.

Bottom line

The current implementation proves that direct transport works on the tested network. The remaining failure is automatic discovery caused by multicast filtering. The cleanest campus-scale solution is a campus-local rendezvous service, with direct peer-to-peer traffic retained as the preferred data path.